SPRING & SPRING BOOT
ANNOTATION INTERVIEW HANDBOOK

A High-Impact, Interview-Focused Revision Guide

Covers: Core IoC · Stereotypes · DI · Bean Lifecycle · Spring Boot · MVC & REST
Validation · Exceptions · Transactions · JPA/Hibernate · Security · Scheduling
Caching · Testing · AOP · 75 Q&A · Scenarios · Rapid Fire · Cheat Sheet

---

Prepared for: 2-3 Day Rapid Interview Preparation
Target Companies: Amazon · Microsoft · Oracle · VMware · Deloitte · Accenture · TCS · Infosys · Capgemini

How to Use This Handbook

This handbook is designed to be completed in 2-3 days. It prioritizes the annotations that are actually asked in interviews over exhaustive documentation.

Suggested Schedule

- Day 1: Chapters 1-8 (Core, DI, Beans, Spring Boot, MVC/REST, Validation)
- Day 2: Chapters 9-17 (Exceptions, Transactions, JPA, Security, Scheduling, Caching, Testing, AOP, Advanced)
- Day 3: Final Interview Preparation section — 75 Q&A, Scenarios, Rapid Fire, and the 5-page Cheat Sheet

Each annotation entry follows the same format

- Definition -> Why is it used -> Where is it used -> Syntax -> Interview Tip -> Common Mistake -> Compared With (if applicable)

Look for these recurring boxes

- 🧠 Quick Memory Trick — a fast mnemonic after every major section
- Chapter Quick Revision — key points, mistakes, tips, and confused-annotation pairs at the end of every chapter

Table of Contents

1. Spring Core & IOC

Foundational concepts every interviewer expects you to explain in your own words before diving into annotations.

What is Inversion of Control (IoC)?

IoC is a design principle where the control of creating and managing objects is transferred from your code to the Spring Container. Instead of you writing 'new Service()', the container creates and wires objects for you.

- Solves: tight coupling between classes.
- Benefit: easier testing, loose coupling, centralized object management.

What is the Spring Container?

The container is the core engine that creates beans, wires dependencies, manages their lifecycle, and configures them. Two key interfaces represent it:

Interface	Description
BeanFactory	Basic container; lazy initialization; lightweight.
ApplicationContext	Advanced container; eager initialization; supports events, i18n, AOP integration.

Interview Tip

- Say: 'ApplicationContext extends BeanFactory and is what Spring Boot uses internally.'

What is a Bean?

A bean is simply an object that is created, configured, and managed by the Spring IoC container instead of being manually instantiated with 'new'.

IoC vs Dependency Injection

IoC is the principle; Dependency Injection (DI) is the pattern used by Spring to implement IoC. DI means dependencies are 'injected' into a class rather than the class creating them itself.

Quick Memory Trick

Term	Think Of It As
IoC	Container is the boss, not you
Bean	Any object Spring manages
ApplicationContext	Full-featured container Spring Boot uses

Chapter Quick Revision

Key Points

- IoC inverts object-creation control to the container.
- ApplicationContext is the container Spring Boot auto-creates.
- A 'bean' = any Spring-managed object.

Top Mistakes

- Confusing IoC (principle) with DI (implementation technique).
- Saying BeanFactory and ApplicationContext are the same thing.

Interview Tips

- Always give a one-line definition before an example when asked 'What is IoC?'

Frequently Confused Annotations

- BeanFactory vs ApplicationContext

2. Stereotype Annotations

These mark a class as a Spring-managed bean and give the container a hint about its role.

@Component

Definition

A generic stereotype annotation that marks any Java class as a Spring-managed bean, to be picked up during component scanning.

Why is it used?

- Lets Spring auto-detect and register a class as a bean without XML.
- Use it when a class doesn't fit Service/Repository/Controller roles.

Where is it used?

Utility classes, helper classes, generic beans.

Syntax

```
@Component
public class PriceCalculator {
    // logic
}
```

Interview Tip

- Explain that @Service, @Repository, @Controller are all specializations of @Component (meta-annotated with it).

Common Mistake

- Forgetting @ComponentScan covers the package, so the bean is never registered.

Compared With

Annotation	Special Behavior Added
@Component	None — generic
@Service	None functionally, semantic only
@Repository	Exception translation (DataAccessException)
@Controller	Web request handling

@Service

Definition

A specialization of @Component used to mark the service layer, holding business logic.

Why is it used?

- Improves code readability by clarifying the layer.
- Used by tools/AOP for pointcuts on the service layer.

Where is it used?

Business/service layer classes.

Syntax

```
@Service
public class OrderService {
    public void placeOrder() { }
}
```

Interview Tip

- Mention it carries no extra runtime behavior over @Component — purely semantic clarity.

Common Mistake

- Putting DB access code directly in @Service instead of delegating to @Repository.

@Repository

Definition

A specialization of @Component for the persistence/DAO layer that also enables automatic translation of database exceptions into Spring's DataAccessException hierarchy.

Why is it used?

- Marks DAO layer clearly.
- Converts vendor-specific SQL exceptions into unchecked, consistent Spring exceptions.

Where is it used?

DAO / Repository interfaces and implementation classes.

Syntax

```
@Repository
public interface UserRepository extends JpaRepository<User, Long> {
}
```

Interview Tip

- This is the ONE stereotype with real extra behavior — exception translation. Interviewers love this distinction.

Common Mistake

- Assuming @Repository is required for Spring Data JPA repository interfaces (it isn't — JpaRepository is auto-detected).

@Controller

Definition

Marks a class as a Spring MVC controller that handles incoming web requests and typically returns view names.

Why is it used?

- Used for traditional server-rendered views (JSP/Thymeleaf).
- Works with @ResponseBody per-method if JSON is needed.

Where is it used?

Web layer classes returning views.

Syntax

```
@Controller
public class HomeController {
    @GetMapping("/home")
    public String home() { return "home"; }
}
```

Interview Tip

- Clarify: without @ResponseBody, the returned String is treated as a VIEW name, not response body.

Common Mistake

- Using @Controller for a REST API and forgetting @ResponseBody, causing a 'view not found' error.

@RestController

Definition

A convenience annotation combining @Controller and @ResponseBody — every method's return value is written directly to the HTTP response body (usually as JSON).

Why is it used?

- Simplifies REST API development.
- Avoids adding @ResponseBody to every method.

Where is it used?

REST API controller classes.

Syntax

```
@RestController
public class UserController {
    @GetMapping("/users")
    public List<User> getUsers() { return service.getAll(); }
}
```

Interview Tip

- Say clearly: @RestController = @Controller + @ResponseBody.

Common Mistake

- Trying to return a view name (like 'home.html') from @RestController — it will be sent as body text, not resolved as a view.

Compared With

@Controller	@RestController
Returns view names	Returns data (JSON/XML) directly
Needs @ResponseBody per method for JSON	@ResponseBody applied automatically

---

Comparison: @Component vs @Service vs @Repository vs @Controller

Annotation	Layer	Extra Behavior
@Component	Generic	None
@Service	Business logic	None (semantic)
@Repository	Persistence/DAO	Exception translation
@Controller	Web/MVC	Request handling, view resolution

Quick Memory Trick

Term	Think Of It As
Component	Any Spring object
Service	Business Logic
Repository	Database
Controller	Handles Requests

Chapter Quick Revision

Key Points

- All four are specializations of @Component and are found via @ComponentScan.
- @Repository uniquely adds exception translation.
- @RestController = @Controller + @ResponseBody.

Top Mistakes

- Believing @Service/@Controller change runtime behavior like @Repository does.
- Forgetting @ResponseBody in a plain @Controller REST method.

Interview Tips

- Always mention that these are meta-annotated with @Component when asked 'are they the same?'

Frequently Confused Annotations

- @Controller vs @RestController
- @Component vs @Service vs @Repository vs @Controller

3. Dependency Injection

How Spring supplies a bean's dependencies automatically.

@Autowired

Definition

Tells Spring to automatically resolve and inject a matching bean into a field, constructor, or setter.

Why is it used?

- Removes manual object creation for dependencies.
- Supports field, setter, and constructor injection.

Where is it used?

Fields, constructors, setter methods.

Syntax

```
@Autowired
private OrderService orderService;

// Preferred: constructor injection
public OrderController(OrderService orderService) {
    this.orderService = orderService;
}
```

Interview Tip

- State that from Spring 4.3+, @Autowired is optional on constructors if the class has only one constructor.

Common Mistake

- Using field injection everywhere — makes unit testing harder and hides mandatory dependencies.

@Qualifier

Definition

Used alongside @Autowired to specify exactly which bean to inject when multiple beans of the same type exist.

Why is it used?

- Resolves the 'NoUniqueBeanDefinitionException' ambiguity.
- Lets you pick a bean by its name.

Where is it used?

Fields/parameters, next to @Autowired.

Syntax

```
@Autowired
@Qualifier("paypalGateway")
private PaymentGateway gateway;
```

Interview Tip

- Mention @Qualifier value must match the bean name (class name with lowercase first letter, or explicit @Component("name")).

Common Mistake

- Misspelling the qualifier name, causing a 'no bean found' error instead of ambiguity error.

@Primary

Definition

Marks one bean as the default choice when multiple candidates of the same type exist and no @Qualifier is specified.

Why is it used?

- Provides a sensible default without needing @Qualifier at every injection point.

Where is it used?

On a @Bean method or @Component class.

Syntax

```
@Primary
@Service
public class PaypalGateway implements PaymentGateway { }
```

Interview Tip

- @Qualifier at the injection point always overrides @Primary.

Common Mistake

- Marking two beans of the same type as @Primary — Spring throws an error at startup.

Compared With

@Qualifier	@Primary
Applied at injection point	Applied at bean definition
Explicit per-use selection	Sets a default globally

Constructor Injection vs Field Injection

Constructor Injection	Field Injection
Dependencies are final & immutable	Fields are mutable, no final
Easy to unit test (no Spring container needed)	Requires reflection/Spring context to test
Fails fast if dependency missing	Fails only at runtime when used
Recommended by Spring team	Discouraged for production code

Interview Tip

- Always answer 'why constructor injection?' with: immutability, testability, fail-fast, avoids circular dependency masking.

Quick Memory Trick

Term	Think Of It As
@Autowired	Auto-plug the dependency
@Qualifier	Pick a specific one by name
@Primary	Default choice if no name given

Chapter Quick Revision

Key Points

- Constructor injection is the recommended approach since Spring 4.3.
- @Qualifier overrides @Primary.
- Multiple @Primary beans of same type = startup error.

Top Mistakes

- Overusing field injection in production code.
- Not handling ambiguous bean errors with @Qualifier/@Primary.

Interview Tips

- Be ready to code a 2-bean ambiguity scenario live and resolve it both ways.

Frequently Confused Annotations

- @Qualifier vs @Primary
- @Autowired vs Constructor Injection

4. Bean Lifecycle & Scope

How long a bean lives, and hooks to run code at creation/destruction.

@Scope

Definition

Defines how many instances of a bean are created and how long they live within the container.

Why is it used?

- Default scope (singleton) isn't always correct — e.g., stateful beans need prototype scope.

Where is it used?

On @Component or @Bean definitions.

Syntax

```
@Scope("prototype")
@Component
public class ShoppingCart { }
```

Interview Tip

- List the 5 scopes: singleton, prototype, request, session, application — and know singleton is the default.

Common Mistake

- Injecting a prototype-scoped bean into a singleton and expecting a new instance each call (it won't refresh without extra config like scoped proxies).

Compared With

Scope	Lifetime
singleton	One instance per container (default)
prototype	New instance every time requested
request	One instance per HTTP request (web-aware)
session	One instance per HTTP session (web-aware)

@Lazy

Definition

Defers bean creation until it is first requested, instead of at container startup.

Why is it used?

- Improves startup time for rarely-used or heavy beans.
- Useful for breaking certain circular dependency issues.

Where is it used?

On @Component/@Bean, or at an injection point.

Syntax

```
@Lazy
@Service
public class ReportGenerator { }
```

Interview Tip

- Mention the trade-off: errors in a lazy bean surface later (at first use), not at startup.

Common Mistake

- Overusing @Lazy, which can hide configuration errors until production traffic hits that path.

@PostConstruct

Definition

Marks a method to run once, right after the bean's dependencies have been injected — used for initialization logic.

Why is it used?

- Runs setup code (e.g., cache warm-up, validation) exactly once after DI completes.

Where is it used?

Any Spring-managed bean method with no arguments.

Syntax

```
@PostConstruct
public void init() {
    System.out.println("Bean ready");
}
```

Interview Tip

- Note this comes from Jakarta annotations (javax/jakarta.annotation), not Spring itself.

Common Mistake

- Doing heavy blocking work inside @PostConstruct, delaying application startup.

@PreDestroy

Definition

Marks a method to run just before the bean is destroyed/removed from the container — used for cleanup.

Why is it used?

- Releases resources like DB connections, thread pools, or file handles gracefully.

Where is it used?

Any Spring-managed singleton bean.

Syntax

```
@PreDestroy
public void cleanup() {
    System.out.println("Bean destroyed");
}
```

Interview Tip

- Mention it only fires reliably for singleton-scoped beans — prototype beans aren't fully managed till destruction.

Common Mistake

- Expecting @PreDestroy to run for prototype-scoped beans — Spring doesn't manage their full lifecycle.

Compared With

@PostConstruct	@PreDestroy
Runs after bean initialization	Runs before bean destruction
Setup/warm-up logic	Cleanup/resource-release logic

Bean Lifecycle Flow (interview favorite)

```
1. Instantiate bean (constructor call)
2. Populate properties (Dependency Injection)
3. @PostConstruct / afterPropertiesSet()
4. Bean is ready to use
5. Container shutdown -> @PreDestroy / destroy()
```

Quick Memory Trick

Term	Think Of It As
@PostConstruct	'Welcome' hook after birth
@PreDestroy	'Goodbye' hook before death
@Lazy	Don't create me until needed

Chapter Quick Revision

Key Points

- Singleton is default scope; prototype creates a new instance each time.
- @PostConstruct runs after DI; @PreDestroy runs before shutdown.
- @Lazy delays bean creation to first use.

Top Mistakes

- Assuming @PreDestroy works for prototype beans.
- Confusing @Lazy with @Scope('prototype') — they solve different problems.

Interview Tips

- Be ready to draw the bean lifecycle diagram from memory.

Frequently Confused Annotations

- singleton vs prototype
- @PostConstruct vs @PreDestroy

5. Bean Configuration

Java-based configuration — the modern replacement for XML bean definitions.

@Bean

Definition

Declares a method that produces a bean to be managed by the Spring container — used inside a @Configuration class.

Why is it used?

- Needed when you must configure third-party classes you don't own (can't add @Component to their source).
- Gives full control over object construction logic.

Where is it used?

Methods inside @Configuration classes.

Syntax

```
@Bean
public RestTemplate restTemplate() {
    return new RestTemplate();
}
```

Interview Tip

- Say clearly: use @Bean for third-party/library classes; use @Component for your own classes.

Common Mistake

- Calling another @Bean method directly (a.b()) instead of letting Spring proxy the call — without CGLIB proxying (default) this can create duplicate instances.

@Configuration

Definition

Marks a class as a source of bean definitions — a modern, type-safe replacement for XML configuration files.

Why is it used?

- Groups related @Bean methods together.
- Enables CGLIB proxying so inter-bean method calls still return the singleton.

Where is it used?

Configuration classes, usually one per module/feature.

Syntax

```
@Configuration
public class AppConfig {
    @Bean
    public DataSource dataSource() { ... }
}
```

Interview Tip

- Explain proxyBeanMethods=true (default) is what makes @Bean methods return the same singleton when called from within the class.

Common Mistake

- Using @Component instead of @Configuration for a class full of @Bean methods — loses CGLIB proxying guarantees.

Compared With

@Component	@Bean
Class-level, auto-detected via scanning	Method-level, inside @Configuration

@Component	@Bean
For classes you own	For classes you don't own / need custom construction

---

@Value

Definition

Injects a value from a properties file, environment variable, or a SpEL expression into a field or parameter.

Why is it used?

- Externalizes configuration instead of hardcoding values.

Where is it used?

Fields, constructor/method parameters.

Syntax

```
@Value("${server.port}")
private int port;

@Value("${app.name:DefaultApp}")
private String appName;
```

Interview Tip

- Show you know the ':' default-value syntax in case the property is missing.

Common Mistake

- Using @Value on a static field — Spring cannot inject into static fields directly.

@ComponentScan

Definition

Tells Spring which packages to scan for annotated components (@Component, @Service, etc.).

Why is it used?

- Without it (or @SpringBootApplication, which includes it), beans in other packages won't be detected.

Where is it used?

On a @Configuration or the main application class.

Syntax

```
@ComponentScan(basePackages = "com.example.app")
@Configuration
public class AppConfig { }
```

Interview Tip

- Mention @SpringBootApplication already includes @ComponentScan on the package of the main class and its sub-packages.

Common Mistake

- Placing the main class in a package above sibling packages, causing components outside it to be skipped.

Comparison: @Bean vs @Configuration

@Bean	@Configuration
Declares a single bean	Declares a class as a config source containing @Bean methods
Method-level annotation	Class-level annotation

Quick Memory Trick

Term	Think Of It As
@Bean	Manual object, I control creation
@Configuration	This class is a bean factory blueprint
@Value	Pull a value from properties

Chapter Quick Revision

Key Points

- @Bean = method-level, third-party objects; @Component = class-level, your own classes.
- @Configuration enables CGLIB proxying for consistent singleton behavior.
- @Value reads from application.properties/yml with optional defaults.

Top Mistakes

- Using @Component on a class full of @Bean factory methods.
- Hardcoding config values instead of externalizing with @Value/@ConfigurationProperties.

Interview Tips

- Be ready to explain proxyBeanMethods and why it matters for inter-bean calls.

Frequently Confused Annotations

- @Bean vs @Configuration
- @Component vs @Bean

6. Spring Boot Core Annotations

The annotations that make Spring Boot 'auto-magical' and reduce boilerplate configuration.

@SpringBootApplication

Definition

A convenience meta-annotation combining @Configuration, @EnableAutoConfiguration, and @ComponentScan — the single annotation placed on the main class.

Why is it used?

- Bootstraps the entire application with sensible defaults in one line.

Where is it used?

The main application class only.

Syntax

```
@SpringBootApplication
public class MyApp {
    public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args);
    }
}
```

Interview Tip

- Be ready to name the 3 annotations it bundles — this is asked in almost every interview.

Common Mistake

- Placing the main class too deep in the package hierarchy so @ComponentScan misses sibling packages.

@EnableAutoConfiguration

Definition

Tells Spring Boot to automatically configure beans based on the jars present on the classpath and defined properties.

Why is it used?

- Removes manual configuration for common setups (e.g., DataSource is auto-configured if a DB driver is on the classpath).

Where is it used?

Rarely used alone — bundled inside @SpringBootApplication.

Syntax

```
@EnableAutoConfiguration
@Configuration
public class LegacyConfig { }
```

Interview Tip

- Mention @SpringBootApplication already includes this — you rarely add it manually.

Common Mistake

- Not knowing you can exclude specific auto-configurations: @EnableAutoConfiguration(exclude = DataSourceAutoConfiguration.class).

@ConfigurationProperties

Definition

Binds a whole group of external properties (from application.yml/properties) to a strongly-typed Java object.

Why is it used?

- Cleaner than dozens of @Value fields when config has many related properties.

Where is it used?

POJO classes, often combined with @Component or @EnableConfigurationProperties.

Syntax

```
@ConfigurationProperties(prefix = "app.mail")
@Component
public class MailProperties {
    private String host;
    private int port;
    // getters/setters
}
```

Interview Tip

- Mention it supports relaxed binding (app.mail-host / app.mailHost / APP_MAIL_HOST all bind the same).

Common Mistake

- Forgetting getters/setters — property binding relies on them (or a constructor-binding record in newer Spring Boot).

Quick Revision: Other Spring Boot Annotations

@EnableConfigurationProperties · Registers a @ConfigurationProperties class as a bean. *Why: used when the properties POJO isn't itself a @Component.* `Syntax: @EnableConfigurationProperties(MailProperties.class)`

@ConditionalOnProperty · Enables a bean only if a given property has a specific value. *Why: feature toggles via properties.* `Syntax: @ConditionalOnProperty(name="feature.x", havingValue="true")`

@ConditionalOnMissingBean · Registers a bean only if no other bean of that type exists. *Why: used heavily inside Spring Boot's own auto-configuration classes.* `Syntax: @ConditionalOnMissingBean(DataSource.class)`

Quick Memory Trick

Term	Think Of It As
@SpringBootApplication	The 3-in-1 starter switch
@EnableAutoConfiguration	Guess my beans from the classpath
@ConfigurationProperties	Bulk-bind a properties group

Chapter Quick Revision

Key Points

- @SpringBootApplication = @Configuration + @EnableAutoConfiguration + @ComponentScan.
- @ConfigurationProperties binds groups of related properties to a POJO.
- Auto-configuration reacts to the classpath and existing beans.

Top Mistakes

- Manually adding @EnableAutoConfiguration when @SpringBootApplication already includes it.
- Using @Value for 10+ related properties instead of @ConfigurationProperties.

Interview Tips

- Know how to exclude an auto-configuration class by name.

Frequently Confused Annotations

- @Value vs @ConfigurationProperties

7. Spring MVC & REST Annotations

Annotations that map HTTP requests to Java methods and shape request/response data.

@RequestMapping

Definition

The general-purpose annotation to map HTTP requests (any method: GET/POST/PUT/DELETE) to a handler method or class.

Why is it used?

- Base mapping annotation; the @GetMapping family are shortcuts built on top of it.

Where is it used?

Controller classes (base path) and methods.

Syntax

```
@RequestMapping("/api/users")
@RestController
public class UserController {
    @RequestMapping(value="/{id}", method=RequestMethod.GET)
    public User get(@PathVariable Long id) { ... }
}
```

Interview Tip

- Mention it can be placed at class level to define a common base path for all methods.

Common Mistake

- Using @RequestMapping without specifying method=, unintentionally allowing GET/POST/etc. all on the same path.

@GetMapping

Definition

A shortcut for @RequestMapping(method = RequestMethod.GET) — used to read/fetch data.

Why is it used?

- Cleaner, more readable REST endpoint definitions.

Where is it used?

Controller methods that retrieve data.

Syntax

```
@GetMapping("/users/{id}")
public User getUser(@PathVariable Long id) { ... }
```

Interview Tip

- Know the full family: @GetMapping, @PostMapping, @PutMapping, @DeleteMapping, @PatchMapping.

Common Mistake

- Using @GetMapping for an operation that changes server state (violates REST semantics/idempotency expectations).

@PostMapping

Definition

A shortcut for @RequestMapping(method = RequestMethod.POST) — used to create new resources.

Why is it used?

- Standard REST convention for resource creation.

Where is it used?

Controller methods that create data.

Syntax

```
@PostMapping("/users")
public User create(@RequestBody User user) { ... }
```

Interview Tip

- Mention POST is not idempotent — calling it twice can create two resources, unlike PUT.

Common Mistake

- Forgetting @RequestBody, so Spring tries to bind the JSON payload as request params instead of the object.

@RequestBody

Definition

Binds the HTTP request body (typically JSON) to a Java object using an HttpMessageConverter (Jackson by default).

Why is it used?

- Automatically deserializes incoming JSON into POJOs.

Where is it used?

Method parameters in @PostMapping/@PutMapping/@PatchMapping handlers.

Syntax

```
@PostMapping("/users")
public User create(@RequestBody User user) { ... }
```

Interview Tip

- Mention it relies on Jackson (jackson-databind) being on the classpath, which Spring Boot includes by default.

Common Mistake

- Using @RequestBody on a GET request — GET typically has no body and this leads to errors.

@ResponseBody

Definition

Tells Spring to write the method's return value directly into the HTTP response body instead of resolving it as a view name.

Why is it used?

- Core mechanism behind returning JSON from @Controller classes.

Where is it used?

Methods in @Controller classes (redundant in @RestController).

Syntax

```
@Controller
public class ApiController {
    @ResponseBody
    @GetMapping("/ping")
    public String ping() { return "pong"; }
}
```

Interview Tip

- State clearly that @RestController already implies @ResponseBody on every method.

Common Mistake

- Adding @ResponseBody unnecessarily in an already-@RestController class (harmless but shows lack of understanding).

@RequestParam

Definition

Binds a query-string parameter (?key=value) or form field from the request to a method parameter.

Why is it used?

- Used for optional filters, pagination, search terms, etc.

Where is it used?

Controller method parameters.

Syntax

```
@GetMapping("/search")
public List<User> search(@RequestParam(required=false) String name) { ... }
```

Interview Tip

- Mention 'required' and 'defaultValue' attributes to handle optional params gracefully.

Common Mistake

- Not setting required=false for optional params, causing a 400 error when the param is absent.

@PathVariable

Definition

Extracts a value from the URI template itself (e.g., /users/{id}) and binds it to a method parameter.

Why is it used?

- Used for identifying a specific resource in RESTful URLs.

Where is it used?

Controller method parameters, matching a {placeholder} in the mapping path.

Syntax

```
@GetMapping("/users/{id}")
public User getUser(@PathVariable Long id) { ... }
```

Interview Tip

- If the variable name differs from the placeholder, show you know @PathVariable("userId").

Common Mistake

- Mismatched placeholder name and parameter name without explicitly specifying the value in the annotation.

Compared With

@RequestParam	@PathVariable
From query string (?id=5)	From URI path (/users/5)
Good for optional/filter data	Good for identifying a specific resource

---

Quick Revision: Other MVC Annotations

@PutMapping · Shortcut for full-resource update via HTTP PUT. *Why: idempotent full replace of a resource.* `Syntax: @PutMapping("/users/{id}")`

@PatchMapping · Shortcut for partial-resource update via HTTP PATCH. *Why: update only specific fields.* `Syntax: @PatchMapping("/users/{id}")`

@DeleteMapping · Shortcut for HTTP DELETE to remove a resource. *Why: standard REST delete operation.* `Syntax: @DeleteMapping("/users/{id}")`

@ModelAttribute · Binds form/query data into a model object, and can pre-populate the Model for views. *Why: classic form-binding in server-rendered MVC apps.* `Syntax: public String save(@ModelAttribute User user)`

@CrossOrigin · Enables CORS for a controller/method for specified origins. *Why: allow frontend apps on a different domain/port to call the API.* `Syntax: @CrossOrigin(origins="http://localhost:3000")`

Comparison: @RequestBody vs @ModelAttribute

@RequestBody	@ModelAttribute
Reads JSON/XML body via message converters	Reads form fields/query params into an object
Common in REST APIs	Common in traditional MVC form submissions

Comparison: PUT vs PATCH

PUT	PATCH
Replaces the entire resource	Updates only the given fields
Idempotent	May or may not be idempotent

Quick Memory Trick

Term	Think Of It As
@RequestParam	?query=string params
@PathVariable	/path/{segments}
@RequestBody	Incoming JSON payload
@ResponseBody	Outgoing JSON payload

Chapter Quick Revision

Key Points

- @GetMapping/@PostMapping/etc. are shortcuts for @RequestMapping(method=...).
- @RequestParam reads query params; @PathVariable reads URI segments.
- @RestController = @Controller + @ResponseBody on every method.

Top Mistakes

- Forgetting @RequestBody on POST/PUT handlers.
- Using GET for state-changing operations.

Interview Tips

- Be ready to design a full CRUD controller live — this is a very common live-coding ask.

Frequently Confused Annotations

- @RequestParam vs @PathVariable
- @RequestBody vs @ModelAttribute
- PUT vs PATCH

8. Validation Annotations

Enforcing data correctness on incoming requests using Bean Validation (JSR-380).

@Valid

Definition

Triggers standard Java Bean Validation (JSR-380) on an object — a standard Java/Jakarta EE annotation, not Spring-specific.

Why is it used?

- Ensures request payloads meet constraints (e.g., @NotNull, @Size) before hitting business logic.

Where is it used?

Controller method parameters, and for nested object validation.

Syntax

```
@PostMapping("/users")
public User create(@Valid @RequestBody User user) { ... }
```

Interview Tip

- Mention it comes from javax.validation / jakarta.validation, standardized across Java EE, not just Spring.

Common Mistake

- Forgetting to add a @ControllerAdvice handler for MethodArgumentNotValidException, resulting in an ugly default 400 response.

@Validated

Definition

A Spring-specific variant of @Valid that additionally supports validation groups and can be applied at the class level for method parameter validation.

Why is it used?

- Needed for validation groups (e.g., different rules for Create vs Update) and for validating @RequestParam/@PathVariable.

Where is it used?

Class level (enables method validation) or parameter level with groups.

Syntax

```
@Validated
@RestController
public class UserController {
    @GetMapping("/users")
    public List<User> list(@RequestParam @Min(1) int page) { ... }
}
```

Interview Tip

- Say @Validated is Spring's own annotation while @Valid is the standard Bean Validation one — @Validated adds groups support.

Common Mistake

- Using @Valid when you actually need validation groups — @Valid does not support groups.

Compared With

@Valid	@Validated
Standard Java/Jakarta annotation	Spring-specific annotation
No support for validation groups	Supports validation groups
Works on nested objects	Works at class level for param validation too

Quick Revision: Field-Level Constraint Annotations

@NotNull · Value must not be null. *Why: mandatory fields.* `Syntax: @NotNull private String name;`

@NotBlank · String must not be null and must contain non-whitespace. *Why: mandatory text fields.* `Syntax: @NotBlank private String email;`

@NotEmpty · Collection/String must not be null or empty (whitespace allowed for strings). *Why: mandatory lists/strings.* `Syntax: @NotEmpty private List<String> tags;`

@Size · Restricts length/size of String, Collection, or array. *Why: min/max length constraints.* `Syntax: @Size(min=2, max=30) private String name;`

@Min / @Max · Restricts numeric value to a minimum/maximum. *Why: numeric range validation.* `Syntax: @Min(18) private int age;`

@Email · Validates a String is a well-formed email address. *Why: email field validation.* `Syntax: @Email private String email;`

@Pattern · Validates a String against a regex. *Why: custom format validation.* `Syntax: @Pattern(regexp="^[A-Z].*") private String code;`

Quick Memory Trick

Term	Think Of It As
@Valid	Standard Java check
@Validated	Spring check + groups
@NotBlank	No empty/whitespace text allowed

Chapter Quick Revision

Key Points

- @Valid is JSR-380 standard; @Validated is Spring-specific with groups support.
- Field-level constraints (@NotNull, @Size, etc.) do the actual rule-checking.
- Validation errors should be caught centrally via @ControllerAdvice.

Top Mistakes

- Not handling MethodArgumentNotValidException, leaking a default stack-trace-like response.
- Reaching for @Validated when plain @Valid would do.

Interview Tips

- Be ready to explain how validation groups work with an interface marker, e.g. OnCreate.class.

Frequently Confused Annotations

- @Valid vs @Validated

9. Exception Handling

Centralizing error handling instead of scattering try/catch across controllers.

@ExceptionHandler

Definition

Marks a method to handle a specific exception type thrown by request-handling methods, returning a custom error response.

Why is it used?

- Avoids repeating try/catch in every controller method.
- Lets you shape a consistent error response body.

Where is it used?

Inside a controller (local) or a @ControllerAdvice class (global).

Syntax

```
@ExceptionHandler(UserNotFoundException.class)
public ResponseEntity<String> handle(UserNotFoundException ex) {
    return ResponseEntity.status(404).body(ex.getMessage());
}
```

Interview Tip

- Mention you can handle multiple exception types in one method: @ExceptionHandler({A.class, B.class}).

Common Mistake

- Defining @ExceptionHandler methods separately in every controller instead of centralizing with @ControllerAdvice.

@ControllerAdvice

Definition

Declares a global, cross-cutting class that applies @ExceptionHandler, @InitBinder, and @ModelAttribute methods to all (or selected) controllers.

Why is it used?

- Centralizes error handling logic across the whole application in one place.

Where is it used?

One class per application (or per module), typically named GlobalExceptionHandler.

Syntax

```
@ControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(Exception.class)
    public ResponseEntity<String> handleAll(Exception ex) {
        return ResponseEntity.status(500).body(ex.getMessage());
    }
}
```

Interview Tip

- For pure REST APIs, mention @RestControllerAdvice (= @ControllerAdvice + @ResponseBody) as the more common real-world choice.

Common Mistake

- Forgetting to also handle validation exceptions (MethodArgumentNotValidException) inside the global handler.

Compared With

@ControllerAdvice	@RestControllerAdvice
Needs @ResponseBody on each handler for JSON	@ResponseBody applied automatically

Quick Revision: Related Exception Annotations

@ResponseStatus · Marks an exception class (or method) with the HTTP status to return when it's thrown/handled. *Why: cleanly ties an exception to an HTTP status code.* `Syntax: @ResponseStatus(HttpStatus.NOT_FOUND) class UserNotFoundException extends RuntimeException {}`

@RestControllerAdvice · @ControllerAdvice + @ResponseBody combined — modern default for REST error handling. *Why: cleanest way to build a global JSON error handler.* `Syntax: @RestControllerAdvice class GlobalExceptionHandler {}`

Quick Memory Trick

Term	Think Of It As
@ExceptionHandler	Catch THIS specific exception
@ControllerAdvice	Catch it EVERYWHERE, globally
@ResponseStatus	Attach an HTTP status to an exception

Chapter Quick Revision

Key Points

- @ExceptionHandler defines how to respond to a specific exception.
- @ControllerAdvice makes handlers global across all controllers.
- @RestControllerAdvice is the REST-friendly combo used in most modern APIs.

Top Mistakes

- Scattering duplicate try/catch blocks instead of centralizing.
- Returning inconsistent error response shapes across different handlers.

Interview Tips

- Design a standard ErrorResponse POJO (timestamp, status, message, path) — interviewers love seeing this structure.

Frequently Confused Annotations

- @ControllerAdvice vs @RestControllerAdvice

10. Transaction Management

Ensuring a group of database operations succeed or fail together.

@Transactional

Definition

Wraps a method (or all public methods of a class) in a database transaction — commits on success, rolls back on a runtime exception.

Why is it used?

- Guarantees atomicity — e.g., debit one account and credit another must both succeed or both fail.

Where is it used?

Service-layer methods/classes performing multiple related DB operations.

Syntax

```
@Transactional
public void transferMoney(Long fromId, Long toId, BigDecimal amount) {
    debit(fromId, amount);
    credit(toId, amount);
}
```

Interview Tip

- Explain: by default, only UNCHECKED exceptions (RuntimeException) trigger rollback, not checked exceptions.

Common Mistake

- Calling a @Transactional method from another method in the SAME class — Spring's proxy is bypassed and the transaction won't apply (self-invocation problem).

Compared With

Propagation	Meaning
REQUIRED (default)	Join existing transaction, or create one if none exists
REQUIRES_NEW	Always start a new, independent transaction
NESTED	Runs within a nested transaction (savepoint)

Banking Example — Scenario Answer

If transferMoney() debits account A but crediting account B throws an exception, @Transactional ensures the debit is rolled back too — the account balances stay consistent. Without it, account A could lose money that never reaches account B.

Key Attributes to Mention

Attribute	Purpose
rollbackFor	Force rollback on checked exceptions too
noRollbackFor	Prevent rollback for specific exceptions
isolation	Controls visibility of uncommitted changes (e.g., READ_COMMITTED)
readOnly	Optimization hint for read-only queries
timeout	Max seconds before the transaction is forced to roll back

Quick Memory Trick

Term	Think Of It As
@Transactional	All-or-nothing DB operations
Self-invocation problem	Calling from within = no proxy = no transaction
rollbackFor	Force rollback for checked exceptions too

Chapter Quick Revision

Key Points

- Default rollback happens only for unchecked exceptions.
- Self-invocation within the same class bypasses the transactional proxy.
- readOnly=true is a useful optimization for pure SELECT operations.

Top Mistakes

- Expecting rollback on checked exceptions without rollbackFor.
- Calling a @Transactional method from a sibling method in the same class.

Interview Tips

- Always mention the banking transfer example — it's the textbook scenario interviewers expect.

Frequently Confused Annotations

- Propagation types (REQUIRED vs REQUIRES_NEW vs NESTED)

11. Spring Data JPA & Hibernate Annotations

Mapping Java classes to database tables and defining relationships.

@Entity

Definition

Marks a Java class as a JPA entity — a class whose instances map to rows in a database table.

Why is it used?

- Required for Hibernate/JPA to manage the class as persistent data.

Where is it used?

Domain/model classes representing database tables.

Syntax

```
@Entity
public class User {
    @Id
    private Long id;
}
```

Interview Tip

- Mention an @Entity class needs a no-arg constructor and a primary key field (@Id) — both are mandatory JPA requirements.

Common Mistake

- Forgetting a no-argument constructor, which Hibernate requires to instantiate entities via reflection.

Compared With

@Entity	@Table
Marks the class as persistable	Optionally customizes table name/schema
Mandatory	Optional — defaults to class name if omitted

@Id

Definition

Marks the field that represents the primary key of the entity.

Why is it used?

- JPA needs to know which field uniquely identifies each row.

Where is it used?

Exactly one field per @Entity class.

Syntax

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

Interview Tip

- Composite keys use @EmbeddedId or @IdClass instead of a single @Id — good to mention if asked about multi-column keys.

Common Mistake

- Missing @Id entirely — Hibernate throws a MappingException at startup.

@GeneratedValue

Definition

Specifies the strategy used to automatically generate primary key values.

Why is it used?

- Avoids manually assigning unique IDs.

Where is it used?

Alongside @Id.

Syntax

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

Interview Tip

- Know the 4 strategies: IDENTITY (DB auto-increment), SEQUENCE, TABLE, AUTO (Hibernate picks).

Common Mistake

- Using AUTO in production without knowing which underlying strategy Hibernate actually picks for your DB — can cause portability surprises.

@Column

Definition

Customizes the mapping between an entity field and its database column (name, length, nullable, unique, etc.).

Why is it used?

- Needed when the field name differs from the column name, or extra constraints are required.

Where is it used?

Entity fields.

Syntax

```
@Column(name = "user_name", nullable = false, length = 100)
private String username;
```

Interview Tip

- Mention that without @Column, Hibernate defaults to the field name as the column name.

Common Mistake

- Forgetting nullable=false when a business rule requires the field to be mandatory at the DB level too.

@OneToMany

Definition

Defines a one-to-many relationship — one entity instance is associated with a collection of another entity.

Why is it used?

- Models real-world parent-child relationships (e.g., one Customer has many Orders).

Where is it used?

The 'one' side of the relationship, typically holding a List/Set.

Syntax

```
@OneToMany(mappedBy = "customer", cascade = CascadeType.ALL)
private List<Order> orders;
```

Interview Tip

- Always mention 'mappedBy' — it marks this as the inverse (non-owning) side of the relationship.

Common Mistake

- Forgetting FetchType.LAZY is the default for @OneToMany but not for @ManyToOne (which is EAGER by default) — a classic interview trap.

@ManyToOne

Definition

Defines a many-to-one relationship — many entity instances refer to a single instance of another entity.

Why is it used?

- Represents the child/owning side of a relationship, e.g., many Orders belong to one Customer.

Where is it used?

The 'many' side, holding a single reference field.

Syntax

```
@ManyToOne
@JoinColumn(name = "customer_id")
private Customer customer;
```

Interview Tip

- State clearly: @ManyToOne is the OWNING side and holds the foreign key column.

Common Mistake

- Not setting fetch = FetchType.LAZY explicitly — default EAGER fetching can cause performance issues (N+1 queries) at scale.

Compared With

@OneToMany	@ManyToOne
Inverse side (usually 'mappedBy')	Owning side (holds foreign key)
Default fetch: LAZY	Default fetch: EAGER

@OneToOne

Definition

Defines a strict one-to-one relationship between two entities.

Why is it used?

- Used to split a large table into two, or model exclusive relationships (e.g., User has one Passport).

Where is it used?

Either side of the relationship, one side owns the foreign key.

Syntax

```
@OneToOne
@JoinColumn(name = "passport_id")
private Passport passport;
```

Interview Tip

- Mention the owning side uses @JoinColumn; the inverse side uses mappedBy.

Common Mistake

- Defining @JoinColumn on both sides, creating two foreign key columns instead of one shared relationship.

@ManyToMany

Definition

Defines a many-to-many relationship, typically implemented via a join table.

Why is it used?

- Models relationships like Students enrolled in many Courses, and Courses having many Students.

Where is it used?

Both sides of the relationship; one side is often marked 'owning'.

Syntax

```
@ManyToMany
@JoinTable(name = "student_course",
  joinColumns = @JoinColumn(name = "student_id"),
  inverseJoinColumns = @JoinColumn(name = "course_id"))
private List<Course> courses;
```

Interview Tip

- For extra columns on the join table (like enrollment date), mention you'd model it as its own @Entity instead.

Common Mistake

- Using @ManyToMany when the join table needs extra attributes — should be refactored into two @OneToMany/@ManyToOne relationships with a dedicated entity.

@JoinColumn

Definition

Specifies the foreign key column used to join two entities in a relationship.

Why is it used?

- Customizes the FK column name instead of relying on Hibernate's default naming.

Where is it used?

On the owning side of @ManyToOne, @OneToOne, or @ManyToMany relationships.

Syntax

```
@ManyToOne
@JoinColumn(name = "customer_id", nullable = false)
private Customer customer;
```

Interview Tip

- Clarify @JoinColumn marks the OWNING side of the relationship — the side with the foreign key.

Common Mistake

- Adding @JoinColumn on the mappedBy (inverse) side too, creating a conflicting/duplicate mapping.

Quick Revision: Other JPA Annotations

@Table · Customizes the table name/schema for an entity. *Why: when class name != table name.* `Syntax: @Table(name="app_users")`

@Transient · Excludes a field from persistence entirely. *Why: computed/derived fields that shouldn't be stored.* `Syntax: @Transient private int age;`

@Embeddable / @Embedded · Embeds a reusable value-type object's fields into the owning table. *Why: e.g., an Address object reused across entities.* `Syntax: @Embedded private Address address;`

@Enumerated · Maps a Java enum to a column, as STRING or ORDINAL. *Why: avoid storing fragile ordinal ints — prefer EnumType.STRING.* `Syntax: @Enumerated(EnumType.STRING) private Status status;`

@Lob · Maps a large object (CLOB/BLOB) field, e.g. long text or binary data. *Why: storing big text/binary content.* `Syntax: @Lob private String description;`

Comparison: @OneToMany vs @ManyToOne

@OneToMany	@ManyToOne
Parent side, usually inverse (mappedBy)	Child side, owning, holds FK via @JoinColumn
Default fetch = LAZY	Default fetch = EAGER

Comparison: @Entity vs @Table

@Entity	@Table
Required — marks class as persistent	Optional — customizes table name/schema

Quick Memory Trick

Term	Think Of It As
@OneToMany	Parent holding a list of children
@ManyToOne	Child pointing to one parent (FK here)
mappedBy	'I am not the owner, look over there'

Chapter Quick Revision

Key Points

- @ManyToOne is the owning side by default (EAGER); @OneToMany is usually inverse (LAZY).
- @JoinColumn goes on the owning side; mappedBy goes on the inverse side.
- Prefer EnumType.STRING over ORDINAL for enum columns.

Top Mistakes

- Mixing up which side owns the foreign key.
- Leaving @ManyToOne as EAGER by default in large object graphs, causing N+1 query issues.

Interview Tips

- Be ready to draw a quick ER-style relationship (Customer -> Orders) and map it with correct annotations live.

Frequently Confused Annotations

- @OneToMany vs @ManyToOne
- @Entity vs @Table

12. Spring Security Annotations

Method-level and configuration-level access control.

@PreAuthorize

Definition

Evaluates a SpEL security expression BEFORE a method executes, and blocks the call if it returns false.

Why is it used?

- Enables fine-grained, expression-based method security (roles, authorities, custom logic).

Where is it used?

Service or controller methods needing role/permission checks.

Syntax

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteUser(Long id) { ... }
```

Interview Tip

- Mention it requires @EnableMethodSecurity (Spring Security 6+) or @EnableGlobalMethodSecurity(prePostEnabled=true) in older versions.

Common Mistake

- Forgetting to enable method security globally, so @PreAuthorize is silently ignored.

Compared With

@PreAuthorize	@Secured
SpEL expressions — flexible (hasRole, hasAuthority, custom beans)	Simple role-name based only
Can reference method arguments	Cannot reference arguments

Quick Revision: Other Security Annotations

@Secured · Restricts method access to users with specific roles — simpler, older alternative to @PreAuthorize. *Why: basic role checks without SpEL.* `Syntax: @Secured("ROLE_ADMIN")`

@RolesAllowed · JSR-250 standard annotation for role-based access control. *Why: Java EE standard alternative, portable across frameworks.* `Syntax: @RolesAllowed("ADMIN")`

@PostAuthorize · Evaluates a security expression AFTER method execution, can check the return value. *Why: authorize based on the result being returned.* `Syntax: @PostAuthorize("returnObject.owner == authentication.name")`

@EnableWebSecurity · Enables Spring Security's web security configuration. *Why: required to activate a custom SecurityFilterChain setup.* `Syntax: @EnableWebSecurity class SecurityConfig {}`

@EnableMethodSecurity · Enables method-level security annotations like @PreAuthorize/@PostAuthorize (Spring Security 6+). *Why: modern replacement for @EnableGlobalMethodSecurity.* `Syntax: @EnableMethodSecurity class SecurityConfig {}`

Quick Memory Trick

Term	Think Of It As
@PreAuthorize	Check BEFORE letting you in
@PostAuthorize	Check AFTER seeing the result
@Secured	Simple bouncer, role name only

Chapter Quick Revision

Key Points

- @PreAuthorize supports full SpEL; @Secured only supports simple role names.
- Method security must be explicitly enabled via @EnableMethodSecurity.
- @PostAuthorize can inspect the return value for authorization decisions.

Top Mistakes

- Forgetting to enable method security and assuming annotations 'just work'.
- Using @Secured when argument-based logic (needs SpEL) is required.

Interview Tips

- Be ready to write a @PreAuthorize expression referencing a method parameter, e.g. hasRole('ADMIN') or #id == authentication.principal.id.

Frequently Confused Annotations

- @PreAuthorize vs @Secured

13. Scheduling & Async

Running background jobs and non-blocking method calls.

@Scheduled

Definition

Marks a method to run automatically at fixed intervals or according to a cron expression.

Why is it used?

- Automates recurring jobs like cleanup tasks, report generation, or polling.

Where is it used?

Methods in Spring-managed beans, with no return value expected to be used.

Syntax

```
@Scheduled(cron = "0 0 * * * *")
public void hourlyJob() { ... }

@Scheduled(fixedRate = 5000)
public void everyFiveSeconds() { ... }
```

Interview Tip

- Know the 3 options: fixedRate (start-to-start interval), fixedDelay (end-to-start interval), and cron.

Common Mistake

- Forgetting to add @EnableScheduling on a @Configuration class — @Scheduled silently does nothing without it.

Compared With

fixedRate	fixedDelay
Next run starts N ms after PREVIOUS START	Next run starts N ms after PREVIOUS run COMPLETES

@Async

Definition

Runs the annotated method in a separate thread, returning control to the caller immediately.

Why is it used?

- Improves throughput for slow, non-blocking-safe operations (emails, notifications, logging).

Where is it used?

Service methods that don't need to block the calling thread.

Syntax

```
@Async
public CompletableFuture<Void> sendEmail(String to) {
    // send email
    return CompletableFuture.completedFuture(null);
}
```

Interview Tip

- Mention @EnableAsync is required, and that calling an @Async method from within the same class won't run asynchronously (same proxy limitation as @Transactional).

Common Mistake

- Expecting exceptions from a void @Async method to propagate to the caller — they don't; they must be handled via an AsyncUncaughtExceptionHandler.

Quick Revision: Related Annotations

@EnableScheduling · Enables detection of @Scheduled methods application-wide. *Why: required for @Scheduled to work at all.* `Syntax: @EnableScheduling class AppConfig {}`

@EnableAsync · Enables detection of @Async methods application-wide. *Why: required for @Async to work at all.* `Syntax: @EnableAsync class AppConfig {}`

Quick Memory Trick

Term	Think Of It As
@Scheduled	Run me on a timer/cron
@Async	Run me on a different thread, don't wait
Self-invocation issue	Both need proxy — calling from within the class fails silently

Chapter Quick Revision

Key Points

- @Scheduled needs @EnableScheduling; @Async needs @EnableAsync.
- fixedRate measures from start-to-start; fixedDelay measures from end-to-start.
- Both are proxy-based and subject to the self-invocation limitation.

Top Mistakes

- Forgetting the @Enable* annotation and wondering why nothing runs.
- Calling an @Async method from a sibling method in the same class.

Interview Tips

- Be ready to explain how thread pools are configured for @Async via a custom Executor bean.

Frequently Confused Annotations

- fixedRate vs fixedDelay

14. Caching

Avoiding repeated expensive computation or DB calls by storing results.

@Cacheable

Definition

Caches the return value of a method; on subsequent calls with the same arguments, the cached value is returned instead of re-executing the method.

Why is it used?

- Reduces load on databases/external services for frequently-requested, rarely-changing data.

Where is it used?

Service methods with expensive, deterministic reads.

Syntax

```
@Cacheable("users")
public User getUserById(Long id) {
    return userRepository.findById(id).orElseThrow();
}
```

Interview Tip

- Mention @EnableCaching is required, and the cache key defaults to the method arguments unless a custom 'key' SpEL is given.

Common Mistake

- Caching a method whose result changes frequently, serving stale data to users.

Compared With

@Cacheable	@CachePut	@CacheEvict
Reads from cache if present, else calls method & stores	Always calls the method AND updates the cache	Removes an entry (or all entries) from the cache

---

Quick Revision: Other Caching Annotations

@CachePut · Always executes the method and updates the cache with the new result. *Why: used to keep cache fresh after an update operation.* `Syntax: @CachePut(value="users", key="#user.id")`

@CacheEvict · Removes one or all entries from a cache. *Why: used after delete operations to avoid stale cache reads.* `Syntax: @CacheEvict(value="users", key="#id")`

@EnableCaching · Activates Spring's annotation-driven cache management. *Why: required for @Cacheable/@CachePut/@CacheEvict to function.* `Syntax: @EnableCaching class AppConfig {}`

@CacheConfig · Class-level annotation to share common cache settings (like cache name) across methods. *Why: reduces repetition of the same cache name in every annotation.* `Syntax: @CacheConfig(cacheNames="users")`

Comparison: @Cacheable vs @CachePut vs @CacheEvict

Annotation	Executes Method?	Cache Action
@Cacheable	Only if cache miss	Reads from cache, or populates on miss
@CachePut	Always	Always overwrites cache entry
@CacheEvict	Always (unless beforeInvocation)	Removes entry/entries

Quick Memory Trick

Term	Think Of It As
@Cacheable	Check cache first, save a trip

Term	Think Of It As
@CachePut	Always go, then update cache
@CacheEvict	Throw the old cache entry away

Chapter Quick Revision

Key Points

- @Cacheable skips method execution on a cache hit; @CachePut never skips.
- @CacheEvict is typically paired with update/delete operations to keep cache correct.
- @EnableCaching must be present for any of these to work.

Top Mistakes

- Using @Cacheable on an update method (stale writes never reach the cache).
- Forgetting to evict related cache entries after a delete.

Interview Tips

- Be ready to explain cache key generation via SpEL, e.g. key = "#id".

Frequently Confused Annotations

- @Cacheable vs @CachePut vs @CacheEvict

15. Testing Annotations

Slicing the Spring context appropriately for fast, focused tests.

@SpringBootTest

Definition

Loads the full Spring application context for integration testing — the heaviest but most complete test annotation.

Why is it used?

- Used when you need to test how multiple layers work together end-to-end.

Where is it used?

Integration test classes.

Syntax

```
@SpringBootTest
class OrderServiceIntegrationTest {
    @Autowired
    private OrderService orderService;
}
```

Interview Tip

- Mention webEnvironment options (MOCK, RANDOM_PORT, DEFINED_PORT, NONE) for controlling how the web server is started.

Common Mistake

- Using @SpringBootTest for every test class — slows down the test suite drastically compared to slice tests.

Compared With

@SpringBootTest	@WebMvcTest	@DataJpaTest
Full context, slow	Web layer only, fast	JPA layer only, fast

@WebMvcTest

Definition

Loads only the web layer (controllers, filters, JSON converters) for fast, focused MVC testing — services/repositories are NOT loaded.

Why is it used?

- Isolates controller logic and HTTP behavior without needing the whole app context.

Where is it used?

Controller unit tests, usually combined with @MockBean for service dependencies.

Syntax

```
@WebMvcTest(UserController.class)
class UserControllerTest {
    @Autowired
    private MockMvc mockMvc;
    @MockBean
    private UserService userService;
}
```

Interview Tip

- Mention MockMvc is auto-configured and used to simulate HTTP requests without starting a real server.

Common Mistake

- Forgetting to @MockBean the service layer — @WebMvcTest won't auto-wire real service/repository beans.

@DataJpaTest

Definition

Loads only the JPA-related components (repositories, entity manager) with an in-memory database by default, for fast repository-layer testing.

Why is it used?

- Verifies repository queries and entity mappings without starting the full application.

Where is it used?

Repository-layer test classes.

Syntax

```
@DataJpaTest
class UserRepositoryTest {
    @Autowired
    private UserRepository userRepository;
}
```

Interview Tip

- Mention it's transactional by default and rolls back after each test automatically.

Common Mistake

- Assuming it tests against your production database — by default it swaps in an embedded in-memory DB unless configured otherwise.

Quick Revision: Other Testing Annotations

@MockBean · Replaces a real bean in the Spring context with a Mockito mock. *Why: isolate the class under test from its real dependencies.* `Syntax: @MockBean private UserService userService;`

@Mock · Plain Mockito annotation to create a mock object (no Spring context involved). *Why: pure unit tests without loading Spring at all.* `Syntax: @Mock private UserService userService;`

@InjectMocks · Injects @Mock fields into the object under test. *Why: wires mocks into the class being tested, Mockito-only.* `Syntax: @InjectMocks private UserController controller;`

@Test · Marks a method as a JUnit test case. *Why: the fundamental JUnit annotation for any test method.* `Syntax: @Test void shouldReturnUser() { ... }`

@BeforeEach / @AfterEach · Runs setup/teardown code before/after every test method (JUnit 5). *Why: reset state between tests.* `Syntax: @BeforeEach void setUp() { ... }`

Quick Memory Trick

Term	Think Of It As
@SpringBootTest	Full app, slow, integration
@WebMvcTest	Just the web layer, fast
@DataJpaTest	Just the JPA layer, fast
@MockBean	Fake bean inside Spring context

Chapter Quick Revision

Key Points

- Slice tests (@WebMvcTest, @DataJpaTest) load only the relevant layer and are much faster than @SpringBootTest.
- @MockBean replaces a Spring bean; @Mock/@InjectMocks are plain Mockito with no Spring context.
- @DataJpaTest is transactional and rolls back automatically.

Top Mistakes

- Overusing @SpringBootTest for simple unit-level checks.

- Confusing @MockBean (Spring-aware) with @Mock (plain Mockito).

Interview Tips

- Be ready to justify which test annotation fits a given scenario — this is a very common scenario question.

Frequently Confused Annotations

- @MockBean vs @Mock
- @SpringBootTest vs @WebMvcTest vs @DataJpaTest

16. AOP Annotations (Quick Revision)

Aspect-Oriented Programming — separating cross-cutting concerns like logging, security, and auditing from business logic.

@Aspect · Marks a class as containing cross-cutting logic (advice) to be applied to other beans. *Why: used to group logging/auditing/security logic outside business code.* `Syntax: @Aspect @Component class LoggingAspect {}`

@Before · Runs advice before the matched method executes. *Why: e.g., logging method entry, permission pre-checks.* `Syntax: @Before("execution(* com.app.service.*.*(..))")`

@After · Runs advice after the matched method completes (success or exception). *Why: cleanup logic that must always run.* `Syntax: @After("execution(* com.app.service.*.*(..))")`

@AfterReturning · Runs advice only after successful method completion, with access to the return value. *Why: logging/auditing successful results.* `Syntax: @AfterReturning(pointcut="...", returning="result")`

@AfterThrowing · Runs advice only when the method throws an exception. *Why: centralized error logging/auditing.* `Syntax: @AfterThrowing(pointcut="...", throwing="ex")`

@Around · Wraps the method entirely — can control whether it runs, modify arguments/return value, and measure execution time. *Why: most powerful advice type, e.g. for performance timing.* `Syntax: @Around("execution(* com.app.service.*.*(..))")`

@Pointcut · Defines a reusable expression describing WHICH join points (methods) advice applies to. *Why: avoids repeating the same execution(...) expression in every advice.* `Syntax: @Pointcut("execution(* com.app.service.*.*(..))")`

@EnableAspectJAutoProxy · Enables Spring's AspectJ-style proxy-based AOP support. *Why: required to activate @Aspect-annotated classes.* `Syntax: @EnableAspectJAutoProxy class AppConfig {}`

Advice Order Cheat Sheet

Advice	Runs When
@Before	Before method execution
@Around	Wraps before AND after
@AfterReturning	After success only
@AfterThrowing	After exception only
@After	Always, regardless of outcome (like 'finally')

Quick Memory Trick

Term	Think Of It As
@Aspect	The cross-cutting concern class
@Pointcut	WHERE the advice applies
@Around	Most powerful — wraps the whole call

Chapter Quick Revision

Key Points

- @Around is the most powerful advice — can short-circuit, modify args/return values.
- @Pointcut expressions avoid duplicating 'execution(...)' patterns.
- AOP relies on proxies — same self-invocation limitation as @Transactional/@Async.

Top Mistakes

- Confusing @After (always runs) with @AfterReturning (only on success).

Interview Tips

- Be ready to write a simple @Around advice measuring method execution time with System.currentTimeMillis().

Frequently Confused Annotations

- @Before vs @After vs @Around

17. Less Important / Advanced Annotations (Quick Revision Only)

Lower interview frequency, but good to recognize if they come up.

@Profile · Restricts a bean to only be registered when a specific profile is active (e.g. dev, prod). *Why: environment-specific configuration (different DB per environment).* `Syntax: @Profile("dev") @Bean public DataSource devDataSource() {...}`

@Conditional · Registers a bean only if a custom Condition class evaluates to true. *Why: advanced, highly customizable conditional bean registration.* `Syntax: @Conditional(OnCloudPlatformCondition.class)`

@Import · Manually imports additional @Configuration classes into the current one. *Why: composing configuration from multiple modules.* `Syntax: @Import(SecurityConfig.class)`

@PropertySource · Loads a specific properties file into the Spring Environment. *Why: loading custom-named property files beyond application.properties.* `Syntax: @PropertySource("classpath:custom.properties")`

@Order · Defines the order in which beans (e.g., filters, aspects) are applied relative to each other. *Why: controlling execution order among multiple beans of the same type.* `Syntax: @Order(1) @Component class FirstFilter {}`

@EnableTransactionManagement · Enables Spring's annotation-driven transaction management (@Transactional). *Why: usually auto-enabled by Spring Boot, but relevant in plain Spring.* `Syntax: @EnableTransactionManagement class AppConfig {}`

@Bean(initMethod/destroyMethod) · Alternative to @PostConstruct/@PreDestroy for classes you don't own (via @Bean attributes). *Why: lifecycle hooks for third-party beans without source access.* `Syntax: @Bean(initMethod="start", destroyMethod="stop")`

@ResponseStatus (class-level) · Ties a custom exception class to a specific HTTP status automatically. *Why: cleaner than setting status manually in every handler.* `Syntax: @ResponseStatus(HttpStatus.CONFLICT) class DuplicateUserException {}`

@RequestHeader · Binds an HTTP request header value to a method parameter. *Why: reading auth tokens, custom headers, content negotiation info.* `Syntax: public void m(@RequestHeader("Authorization") String token)`

@CookieValue · Binds an HTTP cookie value to a method parameter. *Why: reading session/tracking cookies.* `Syntax: public void m(@CookieValue("sessionId") String sessionId)`

@InitBinder · Customizes request data binding (e.g., date formats) for a controller. *Why: custom conversion logic for form/query parameters.* `Syntax: @InitBinder void initBinder(WebDataBinder binder) {...}`

@Repeatable annotations (custom) · Lets a custom annotation be applied multiple times to the same element. *Why: advanced meta-annotation usage, rarely asked in depth.* `Syntax: @Repeatable(Schedules.class) @interface Scheduled {}`

@Indexed · Speeds up component scanning at build time by generating a candidate index. *Why: performance optimization for very large codebases.* `Syntax: @Indexed @Component class BigApp {}`

@EventListener · Marks a method as a listener for application events (ApplicationEvent). *Why: decoupled, event-driven communication between beans.* `Syntax: @EventListener public void onOrderPlaced(OrderPlacedEvent e) {...}`

@ConditionalOnClass / @ConditionalOnBean · Registers a bean only if a given class/bean is present on the classpath/context. *Why: core mechanism behind Spring Boot's own auto-configuration.* `Syntax: @ConditionalOnClass(DataSource.class)`

Quick Memory Trick

Term	Think Of It As
@Profile	Bean only for a specific environment
@Import	Pull in another config class
@Order	Who goes first among equals

Chapter Quick Revision

Key Points

- These are rarely the star of an interview but show depth of knowledge if mentioned.
- @Profile and @Conditional* annotations power environment-specific and auto-configuration logic.
- @EventListener enables decoupled, event-driven design between beans.

Top Mistakes

- Spending too much prep time here instead of on Chapters 1-16.

Interview Tips

- Mention 2-3 of these briefly if asked 'what else do you know?' — shows breadth without over-investing prep time.

Frequently Confused Annotations

- @Profile vs @Conditional

Final Interview Preparation — Part 1: Top 75 Questions & Answers

Concise, interview-quality answers (5-10 lines each). Read through once, then use the Rapid Fire section for quick recall drilling.

Q1. What is the difference between IoC and Dependency Injection?

IoC is the broader design principle where the container controls object creation and wiring. Dependency Injection is the specific pattern Spring uses to implement IoC — dependencies are supplied to a class rather than created by it.

Q2. What is the Spring IoC container?

It's the core component (represented by BeanFactory or ApplicationContext) responsible for creating, configuring, wiring, and managing the lifecycle of beans.

Q3. Difference between BeanFactory and ApplicationContext?

BeanFactory is the basic container with lazy loading; ApplicationContext extends it, adds eager loading, event handling, internationalization, and is what Spring Boot uses internally.

Q4. What is a Spring Bean?

Any object whose instantiation, configuration, and lifecycle are managed by the Spring IoC container instead of being created manually with 'new'.

Q5. Difference between @Component, @Service, @Repository, and @Controller?

All are specializations of @Component. @Service marks business logic, @Controller marks web request handlers, and @Repository additionally translates database exceptions into Spring's DataAccessException hierarchy — the only one with real extra behavior.

Q6. Why is @Repository special compared to @Service and @Controller?

It enables automatic exception translation, converting vendor-specific persistence exceptions into a consistent, unchecked Spring DataAccessException hierarchy.

Q7. What is @RestController and how does it differ from @Controller?

@RestController = @Controller + @ResponseBody, meaning every method's return value is written directly into the HTTP response body (typically as JSON) instead of being resolved as a view name.

Q8. How does @Autowired resolve which bean to inject?

By type first; if multiple beans of the same type exist, Spring uses the field/parameter name to match a bean name, or requires @Qualifier/@Primary to disambiguate.

Q9. How do you resolve NoUniqueBeanDefinitionException?

Use @Qualifier to specify the exact bean name at the injection point, or mark one candidate bean as @Primary to set a default.

Q10. Why prefer constructor injection over field injection?

It makes dependencies immutable (final), easier to unit test without a Spring container, and fails fast at object-creation time if a required dependency is missing.

Q11. What does @Primary do?

Marks one bean as the default candidate to inject when multiple beans of the same type exist and no @Qualifier is specified at the injection point.

Q12. What are the Spring bean scopes?

singleton (default, one instance per container), prototype (new instance per request), plus web-aware scopes: request, session, and application.

Q13. What is the default bean scope in Spring?

Singleton — one shared instance per Spring container.

Q14. What does @Lazy do?

Defers bean creation until it's first requested rather than at container startup, improving startup time for rarely-used beans.

Q15. Difference between @PostConstruct and @PreDestroy?

@PostConstruct runs once after dependency injection completes (setup logic); @PreDestroy runs once before the bean is destroyed (cleanup logic). Both are Jakarta annotations, not Spring-specific.

Q16. Why doesn't @PreDestroy fire for prototype beans?

Spring doesn't manage the complete lifecycle of prototype beans after creation — it hands them off, so it never calls destruction callbacks automatically.

Q17. Difference between @Component and @Bean?

@Component is a class-level annotation for classes you own, auto-detected via component scanning. @Bean is a method-level annotation inside a @Configuration class, typically used for third-party classes or objects needing custom construction logic.

Q18. What is the purpose of @Configuration and proxyBeanMethods?

@Configuration marks a class as a source of bean definitions and (by default) uses CGLIB proxying so that calling one @Bean method from another still returns the same managed singleton instance.

Q19. What does @Value do and how do you set a default?

Injects a property, environment variable, or SpEL expression into a field. A default is set using the colon syntax: @Value("${app.name:DefaultApp}").

Q20. What does @ComponentScan do?

Tells Spring which base packages to scan for annotated components; @SpringBootApplication already includes it for the main class's package and sub-packages.

Q21. What three annotations make up @SpringBootApplication?

@Configuration, @EnableAutoConfiguration, and @ComponentScan.

Q22. What is auto-configuration in Spring Boot?

A mechanism where Spring Boot automatically configures beans based on the libraries present on the classpath and existing bean definitions, reducing manual setup.

Q23. How do you exclude a specific auto-configuration?

Using @SpringBootApplication(exclude = DataSourceAutoConfiguration.class) or the equivalent property in application.properties.

Q24. What is @ConfigurationProperties used for?

Binding a whole group of related external properties to a strongly-typed POJO, instead of using many individual @Value fields.

Q25. Difference between @Value and @ConfigurationProperties?

@Value injects a single property/expression per field; @ConfigurationProperties binds a whole prefix-based group of properties to a structured object in one go.

Q26. What is the difference between @RequestMapping and @GetMapping?

@RequestMapping is the general-purpose mapping annotation supporting any HTTP method; @GetMapping is a shortcut specifically for GET requests, built on top of @RequestMapping.

Q27. Difference between @RequestParam and @PathVariable?

@RequestParam extracts values from the query string (?key=value); @PathVariable extracts values embedded directly in the URI path (/users/{id}).

Q28. What does @RequestBody do?

Deserializes the incoming HTTP request body (usually JSON) into a Java object using a message converter such as Jackson.

Q29. What does @ResponseBody do?

Writes the return value of a controller method directly into the HTTP response body instead of resolving it as a view name.

Q30. Why doesn't my @Controller return JSON without @ResponseBody?

Without it, @Controller treats the returned String as a logical view name to resolve (e.g., a Thymeleaf template), not as literal response content.

Q31. Difference between PUT and PATCH?

PUT replaces the entire resource and is idempotent; PATCH updates only the specified fields and may or may not be idempotent depending on implementation.

Q32. What is @Valid and where does it come from?

A standard Java/Jakarta Bean Validation (JSR-380) annotation that triggers constraint checks (like @NotNull, @Size) on an object — not Spring-specific.

Q33. Difference between @Valid and @Validated?

@Valid is the standard Java annotation with no support for validation groups. @Validated is Spring-specific, supports validation groups, and can be applied at the class level to enable method-parameter validation.

Q34. How do you handle validation errors globally?

By adding an @ExceptionHandler for MethodArgumentNotValidException inside a @ControllerAdvice (or @RestControllerAdvice) class, returning a consistent error response.

Q35. What is @ExceptionHandler used for?

Marking a method to handle a specific exception type thrown during request processing, allowing a custom, appropriate error response.

Q36. What is @ControllerAdvice?

A class-level annotation that applies exception handling, data binding, and model attributes globally across some or all controllers.

Q37. Difference between @ControllerAdvice and @RestControllerAdvice?

@RestControllerAdvice combines @ControllerAdvice with @ResponseBody, so exception handler methods automatically return JSON without needing @ResponseBody on each one.

Q38. What is @ResponseStatus used for?

Attaching an HTTP status code directly to a custom exception class (or handler method), so it's returned automatically when that exception occurs.

Q39. Explain @Transactional and the self-invocation problem.

@Transactional wraps a method in a database transaction via a proxy. If a method in the same class calls another @Transactional method internally, the proxy is bypassed, so no transaction is actually applied to the inner call.

Q40. Does @Transactional roll back on all exceptions?

No — by default it rolls back only on unchecked (RuntimeException) exceptions. Checked exceptions require rollbackFor to trigger a rollback.

Q41. What is transaction propagation? Give two examples.

It defines how a transactional method behaves relative to an existing transaction. REQUIRED (default) joins the existing transaction or creates one; REQUIRES_NEW always starts a fresh, independent transaction.

Q42. What does @Entity require at minimum?

A no-argument constructor and exactly one field marked with @Id representing the primary key.

Q43. What are the @GeneratedValue strategies?

IDENTITY (DB auto-increment), SEQUENCE (DB sequence object), TABLE (a table simulating a sequence), and AUTO (Hibernate picks based on the dialect).

Q44. Difference between @OneToMany and @ManyToOne?

@ManyToOne is the owning side, holds the foreign key via @JoinColumn, and defaults to EAGER fetching. @OneToMany is usually the inverse side (mappedBy), holds a collection, and defaults to LAZY fetching.

Q45. Why is default EAGER fetching on @ManyToOne risky?

It can trigger the N+1 query problem — loading a parent list of N children eagerly triggers N additional queries for each associated entity unless fetching is tuned.

Q46. What does mappedBy indicate?

That the current side is NOT the owner of the relationship — the actual foreign key column is managed by the field named in mappedBy on the other entity.

Q47. How would you model a many-to-many relationship that needs extra data on the join?

Avoid @ManyToMany and instead create a dedicated join entity with @OneToMany/@ManyToOne relationships to both sides, allowing extra columns like enrollment date.

Q48. What does @PreAuthorize do and what must be enabled for it to work?

It evaluates a SpEL security expression before a method runs and blocks access if it evaluates to false; it requires @EnableMethodSecurity (or the older @EnableGlobalMethodSecurity) to be active.

Q49. Difference between @PreAuthorize and @Secured?

@PreAuthorize supports full SpEL expressions (roles, authorities, method arguments); @Secured only supports simple role-name checks.

Q50. What does @Scheduled require to function?

The @EnableScheduling annotation on a configuration class; without it, @Scheduled methods are silently never triggered.

Q51. Difference between fixedRate and fixedDelay?

fixedRate starts the next execution a fixed interval after the PREVIOUS execution STARTED; fixedDelay starts the next execution a fixed interval after the PREVIOUS execution FINISHED.

Q52. What does @Async require to function, and what's a common pitfall?

It requires @EnableAsync; a common pitfall is calling an @Async method from within the same class, which bypasses the proxy and runs synchronously.

Q53. Difference between @Cacheable, @CachePut, and @CacheEvict?

@Cacheable skips method execution on a cache hit; @CachePut always executes the method and updates the cache; @CacheEvict removes one or all entries from the cache.

Q54. What annotation is needed to enable caching?

@EnableCaching on a configuration class.

Q55. Difference between @SpringBootTest and @WebMvcTest?

@SpringBootTest loads the entire application context for full integration testing (slower); @WebMvcTest loads only the web layer for fast, focused controller testing, usually paired with @MockBean.

Q56. What does @DataJpaTest do?

Loads only JPA-related components with an embedded in-memory database by default, and rolls back transactions automatically after each test.

Q57. Difference between @MockBean and @Mock?

@MockBean replaces a real bean inside the Spring application context with a mock; @Mock is plain Mockito and doesn't involve Spring at all.

Q58. What is @Aspect used for?

Marking a class that contains cross-cutting advice (logging, security, auditing) to be woven into other beans' method executions.

Q59. What is the most powerful AOP advice type and why?

@Around, because it wraps the entire method call — it can prevent execution, modify arguments, change the return value, and measure timing, unlike the more limited @Before/@After variants.

Q60. What does @Pointcut do?

Defines a reusable expression describing which join points (methods) a piece of advice should apply to, avoiding repetition across multiple advice annotations.

Q61. What is @Profile used for?

Registering a bean only when a specific environment profile (e.g., dev, prod) is active, enabling environment-specific configuration.

Q62. What is @ConditionalOnMissingBean used for?

Registering a bean only if no other bean of that type already exists in the context — heavily used internally by Spring Boot's own auto-configuration classes.

Q63. What is the difference between @Import and @ComponentScan?

@Import explicitly registers specific configuration classes; @ComponentScan automatically discovers and registers annotated classes across a package.

Q64. What is @EventListener used for?

Marking a method to listen for and react to application events, enabling decoupled, event-driven communication between beans.

Q65. How would you design a global error response structure?

A consistent ErrorResponse POJO (timestamp, HTTP status, message, and request path), returned from every handler inside a centralized @RestControllerAdvice.

Q66. What is the N+1 query problem and how do you avoid it?

It occurs when fetching a list of N parent entities triggers one additional query per parent to fetch related child data. It's mitigated with JOIN FETCH queries, @EntityGraph, or by tuning fetch types to LAZY with explicit fetching where needed.

Q67. When would you choose @Bean over @Component?

When you don't own the class's source code (a third-party library class) or need custom construction logic that can't be expressed with a simple class-level annotation.

Q68. What is the purpose of readOnly=true in @Transactional?

It's an optimization hint telling the persistence provider the method only performs reads, allowing certain performance optimizations and preventing accidental writes.

Q69. How does Spring Boot know which DataSource to auto-configure?

By inspecting the classpath for a JDBC driver and relevant starter dependencies, combined with properties like spring.datasource.url, applying conditional auto-configuration accordingly.

Q70. What is the difference between checked and unchecked exceptions in the context of @Transactional?

Unchecked (RuntimeException and its subclasses) trigger automatic rollback by default; checked exceptions do not unless explicitly configured via rollbackFor.

Q71. How do you version a REST API in Spring?

Common approaches include URI versioning (/api/v1/users), request header versioning, or media-type/content-negotiation versioning — the choice depends on team convention rather than a single 'correct' Spring annotation.

Q72. What is the role of Jackson in Spring MVC?

It's the default JSON library used by HttpMessageConverters to automatically serialize Java objects to JSON (@ResponseBody) and deserialize JSON to Java objects (@RequestBody).

Q73. How would you test a service layer method in isolation?

Using plain Mockito (@Mock, @InjectMocks) without loading the Spring context at all, for the fastest possible unit test feedback loop.

Q74. What is the difference between unit tests and integration tests in Spring Boot?

Unit tests isolate a single class using mocks (fast, no Spring context); integration tests (like @SpringBootTest) load some or all of the Spring context to verify components work together correctly.

Q75. Why might you avoid field injection in production code?

It hides mandatory dependencies, makes classes harder to unit test without reflection or a Spring context, and doesn't allow fields to be declared final.

Q76. What's the benefit of using @RestControllerAdvice over per-controller exception handling?

It centralizes all exception-to-response mapping logic in one place, ensuring consistent error response formatting across the entire API surface.

Final Interview Preparation — Part 2: Scenario-Based Questions

These are the 'which annotation would you use and why' style questions common in live interviews.

Q1. You have two implementations of PaymentGateway (Paypal, Stripe). How do you tell Spring which one to inject?

Use @Qualifier("paypalGateway") at the injection point to pick explicitly, or mark one implementation @Primary to set it as the default when no qualifier is given.

Q2. A colleague injects everything using @Autowired on fields. What would you recommend and why?

Recommend constructor injection instead — it makes dependencies immutable and explicit, fails fast if something required is missing, and is far easier to unit test without spinning up a Spring context.

Q3. Your @Transactional method calls another @Transactional method in the same class, but rollback isn't happening as expected. Why?

This is the classic self-invocation problem — Spring's proxy-based AOP is bypassed for internal method calls, so the inner call runs outside any transactional proxy. The fix is to move the inner method to a separate bean, or use AspectJ weaving instead of proxies.

Q4. Explain @Transactional with a banking transfer example.

In transferMoney(from, to, amount), both debit(from) and credit(to) run inside one @Transactional boundary. If credit() throws a RuntimeException after debit() succeeds, the whole transaction rolls back — the debit is undone too, so money is never lost mid-transfer.

Q5. A REST endpoint needs to accept a JSON payload and validate it before saving. Which annotations do you combine?

@PostMapping on the method, @RequestBody to deserialize JSON into a POJO, and @Valid (or @Validated for groups) to trigger Bean Validation constraints before the service layer is called.

Q6. Your @Scheduled job never runs even though the annotation is present. What's the likely cause?

Missing @EnableScheduling on a configuration class — without it, Spring never registers a task scheduler to detect and invoke @Scheduled methods.

Q7. How would you avoid the N+1 query problem when displaying a list of customers with their orders?

Use a JOIN FETCH JPQL query or a Spring Data @EntityGraph to eagerly fetch orders in a single query, instead of relying on default lazy loading that triggers one query per customer.

Q8. Difference between @Bean and @Component in a real project — when would you use each?

@Component is used for classes you own and want auto-detected via scanning (e.g., your own OrderService). @Bean is used inside a @Configuration class for objects you don't own the source of, like a third-party RestTemplate or ObjectMapper, or when construction logic needs custom parameters.

Q9. Multiple beans of the same type exist and the app fails to start with NoUniqueBeanDefinitionException — how do you resolve it, in two ways?

Either add @Qualifier("beanName") at every injection point to be explicit, or mark exactly one implementation as @Primary so it becomes the default when no qualifier is specified.

Q10. You need a controller-only test that doesn't touch the database. Which annotation and setup would you use?

@WebMvcTest(UserController.class) to load only the web layer, combined with @MockBean for the service dependency and MockMvc to simulate HTTP calls without a real server or database.

Final Interview Preparation — Part 3: Rapid Fire (50 Questions)

One-line questions, one-line answers — ideal for last-minute drilling.

#	Question	Answer
1	Default bean scope?	Singleton
2	Annotation for business logic layer?	@Service
3	Annotation adding exception translation?	@Repository
4	@RestController = ?	@Controller + @ResponseBody
5	Recommended injection type?	Constructor injection
6	Resolve ambiguous bean by name?	@Qualifier
7	Default candidate among multiple beans?	@Primary
8	Runs after DI completes?	@PostConstruct
9	Runs before bean destruction?	@PreDestroy
10	Delays bean creation until first use?	@Lazy
11	Method-level bean definition inside @Configuration?	@Bean
12	Marks a class as a config source?	@Configuration
13	Injects a property value?	@Value
14	Binds a group of properties to a POJO?	@ConfigurationProperties
15	3 annotations inside @SpringBootApplication?	@Configuration, @EnableAutoConfiguration, @ComponentScan
16	Reads query string params?	@RequestParam
17	Reads URI path segments?	@PathVariable
18	Deserializes JSON request body?	@RequestBody
19	Writes return value to response body?	@ResponseBody
20	Handles a specific exception?	@ExceptionHandler
21	Global exception handling class?	@ControllerAdvice
22	@ControllerAdvice + @ResponseBody?	@RestControllerAdvice
23	Ties an exception to an HTTP status?	@ResponseStatus
24	Standard Java bean validation trigger?	@Valid
25	Spring validation with groups support?	@Validated
26	Marks a class as a persistent entity?	@Entity
27	Marks the primary key field?	@Id
28	Defines PK generation strategy?	@GeneratedValue
29	Customizes column mapping?	@Column
30	Owning side of a relationship (usually)?	@ManyToOne
31	Default fetch type of @ManyToOne?	EAGER
32	Default fetch type of @OneToMany?	LAZY
33	Marks the inverse side of a relationship?	mappedBy attribute
34	Wraps a method in a DB transaction?	@Transactional
35	Default rollback behavior of @Transactional?	Rolls back only on unchecked exceptions
36	Role-based method security with SpEL?	@PreAuthorize

#	Question	Answer
37	Simple role-only method security?	@Secured
38	Required to enable @Scheduled?	@EnableScheduling
39	Required to enable @Async?	@EnableAsync
40	Interval measured start-to-start?	fixedRate
41	Interval measured end-to-start?	fixedDelay
42	Skips method execution on cache hit?	@Cacheable
43	Always executes and updates cache?	@CachePut
44	Removes cache entries?	@CacheEvict
45	Loads full Spring context for tests?	@SpringBootTest
46	Loads only the web layer for tests?	@WebMvcTest
47	Loads only the JPA layer for tests?	@DataJpaTest
48	Replaces a real bean with a mock in Spring context?	@MockBean
49	Marks a cross-cutting logic class?	@Aspect
50	Most powerful AOP advice type?	@Around
51	Restricts a bean to an active environment?	@Profile

Final 5-Page Revision Cheat Sheet

Highest-yield annotations and concepts only — for the last hour before your interview.

1. Stereotypes

Annotation	One-liner
@Component	Generic Spring-managed bean
@Service	Business logic layer (semantic only)
@Repository	DAO layer + exception translation
@Controller	Web layer, returns views
@RestController	@Controller + @ResponseBody, returns data

2. Dependency Injection

Annotation	One-liner
@Autowired	Auto-inject a matching bean
@Qualifier	Pick a bean by exact name
@Primary	Default bean among duplicates
Constructor Injection	Preferred: immutable, testable, fail-fast

3. Bean Lifecycle & Config

Annotation	One-liner
@Scope	singleton (default) / prototype / request / session
@Lazy	Create bean only when first needed
@PostConstruct / @PreDestroy	Setup / cleanup hooks
@Bean / @Configuration	Manual bean method / config class
@Value / @ConfigurationProperties	Single value / grouped property binding

4. Spring Boot & Web

Annotation	One-liner
@SpringBootApplication	@Configuration + @EnableAutoConfiguration + @ComponentScan
@GetMapping/@PostMapping/...	HTTP-method-specific shortcuts for @RequestMapping
@RequestParam vs @PathVariable	Query string vs URI path
@RequestBody / @ResponseBody	Incoming JSON / outgoing JSON

5. Validation & Errors

Annotation	One-liner
@Valid	Standard JSR-380 validation trigger
@Validated	Spring validation + groups + class-level
@ExceptionHandler	Handle one exception type
@RestControllerAdvice	Global JSON error handling

6. Transactions & Data

Annotation	One-liner
@Transactional	All-or-nothing DB operations; rolls back on RuntimeException by default
@Entity / @Id / @GeneratedValue	Persistent class / primary key / PK strategy
@ManyToOne	Owning side, holds FK, default EAGER
@OneToMany	Inverse side (mappedBy), default LAZY

7. Security, Scheduling, Caching, Testing, AOP

Annotation	One-liner
@PreAuthorize	SpEL-based method security
@Scheduled + @EnableScheduling	Cron/fixed-rate background jobs
@Async + @EnableAsync	Run method on a separate thread
@Cacheable / @CachePut / @CacheEvict	Read cache / always update / remove
@SpringBootTest / @WebMvcTest / @DataJpaTest	Full context / web slice / JPA slice
@Aspect + @Around	Cross-cutting logic, most powerful advice

Top 5 Self-Invocation & Proxy Traps

- @Transactional, @Async, and AOP advice are all proxy-based — calling from within the same class bypasses them.
- @ManyToOne defaults to EAGER; watch for the N+1 query problem in production.
- @PreDestroy does not reliably fire for prototype-scoped beans.
- @Valid has no groups support; use @Validated when you need groups.
- @RestControllerAdvice is almost always preferred over @ControllerAdvice for pure REST APIs.